

PROJET 10

Évaluation et amélioration d'un assistant d'analyses sportives

Rapport de mise en place et d'évaluation du système

Architecture hybride RAG + SQL avec agent PydanticAI

Thomas Auvin

Formation Data Scientist - Openclassroom
2026

1. Introduction

Ce projet avait pour ambition de concevoir, analyser, améliorer et évaluer un système fondé sur un grand modèle de langage (LLM), appliqué à un cas d'usage concret : un assistant d'analyses sportives capable de traiter des questions de nature variée.

Le système devait répondre à deux grandes familles de demandes :

- des questions documentaires, interprétatives ou explicatives, fondées sur des contenus textuels non structurés ;
- des questions statistiques, comparatives ou agrégatives, fondées sur des données tabulaires structurées.

Ce double objectif illustre une problématique classique des systèmes LLM appliqués à des données métier : un simple chatbot génératif ne garantit pas des réponses fiables, sourcées et adaptées au type de question posé. Il fallait donc dépasser une approche purement conversationnelle et construire une architecture plus robuste, à la fois sur le plan fonctionnel, méthodologique et technique.

La démarche adoptée a été progressive : partir d'un système RAG initial, puis le faire évoluer grâce à une logique d'évaluation et d'amélioration continue. Le projet n'a pas seulement visé à produire une application fonctionnelle — il a également porté sur la capacité à mesurer objectivement les performances du système, à identifier ses limites, puis à justifier les choix d'évolution effectués.

2. Contexte et problématique

Le système initial visait à répondre à des questions NBA à partir d'un ensemble hétérogène de sources :

- des documents textuels de type discussion, commentaire ou analyse ;
- des données tabulaires issues d'un fichier Excel regroupant statistiques joueurs, équipes et définitions de métriques.

On pourrait imaginer traiter l'ensemble de ces contenus avec une seule approche RAG : transformer toutes les données en texte, les découper en fragments, les vectoriser et laisser le modèle retrouver les passages pertinents avant de générer une réponse. Cette approche fonctionne relativement bien pour des contenus narratifs ou argumentatifs, mais elle montre rapidement ses limites dès qu'une question nécessite un tri, une agrégation, un filtrage ou une comparaison précise sur des données tabulaires. Elle ne permet pas non plus de distinguer clairement les erreurs dues au retrieval, à la génération, au mauvais outil choisi ou à l'absence d'information exploitable.

La problématique du projet peut donc se formuler ainsi : comment construire un assistant LLM capable de répondre à la fois à des questions documentaires et à des questions statistiques, tout en garantissant un niveau minimal de traçabilité, d'évaluabilité et de robustesse ?

Cette question a conduit à faire évoluer le système selon trois axes :

- structurer l'application pour mieux contrôler les entrées, les sorties et les outils utilisés ;

- distinguer les traitements adaptés aux données textuelles de ceux adaptés aux données structurées ;
 - mettre en place une démarche d'évaluation formelle pour guider les améliorations.
-

3. Objectifs du projet

Le projet poursuivait plusieurs objectifs complémentaires.

Le premier était de mettre à disposition un assistant utilisable, sous la forme d'une interface Streamlit permettant de poser des questions en langage naturel.

Le deuxième consistait à améliorer la qualité des réponses en distinguant deux cas : les questions relevant de la recherche de contexte documentaire, et celles relevant d'une interrogation structurée sur des statistiques.

Le troisième visait à construire une démarche d'évaluation formelle du système — permettant de juger la qualité du projet non à l'intuition, mais à partir de métriques explicites et reproductibles.

Le quatrième objectif portait sur la robustesse d'ingénierie : structuration claire du code, modèles de données explicites, dockerisation, intégration continue minimale et observabilité des exécutions.

4. Version initiale du système

4.1 Architecture initiale

La première version reposait sur une architecture RAG classique :

- une interface Streamlit pour l'interaction utilisateur ;
- un pipeline de chargement et parsing documentaire ;
- un découpage en chunks, une génération d'embeddings via Mistral, et une indexation vectorielle via FAISS ;
- une étape de retrieval sémantique suivie d'une génération de réponse.

Cette architecture offrait un prototype fonctionnel de bout en bout, permettant de vérifier la faisabilité du projet et de tester un pipeline RAG complet avant d'envisager des évolutions.

4.2 Limites identifiées

Trois limites principales sont rapidement apparues. D'abord, toutes les sources étaient ramenées à du texte, ce qui faisait perdre la structure des données tabulaires (statistiques NBA). Ensuite, le système traitait presque toutes les questions selon la même logique linéaire — question → retrieval → génération — sans distinction selon le type de question. Enfin, en l'absence de protocole d'évaluation, il était impossible de diagnostiquer l'origine précise d'une réponse incorrecte : mauvais retrieval, mauvaise génération, mauvais routage ou donnée absente.

Ces constats ont fait de l'évaluation le premier vrai chantier du projet, avant toute modification architecturale.

5. Mise en place de l'évaluation

5.1 Pourquoi évaluer avant d'améliorer

Avant de modifier le système, il était indispensable de répondre à une question simple : qu'est-ce qui ne fonctionne pas, et pourquoi ? Un système RAG ou agentique ne doit pas être jugé uniquement sur quelques réponses observées à la main. Plusieurs dimensions méritent d'être distinguées : qualité du retrieval, pertinence de la réponse, fidélité au contexte, qualité du routage entre les outils.

C'est dans cette logique qu'une évaluation avec RAGAS a été mise en place, accompagnée d'un notebook d'analyse des résultats.

5.2 Runner d'évaluation et métriques suivies

Un script dédié a été construit pour exécuter le système sur un dataset de questions, conserver les traces d'exécution et calculer différentes familles de métriques.

Famille de métriques	Métriques utilisées
Métriques RAGAS	Faithfulness, Answer Relevancy
Métriques de retrieval	Hit@k, Precision@k, Recall@k, MRR
Métriques de routage	Outil appelé, correspondance au type de question

L'objectif n'était pas d'obtenir une note globale, mais de disposer d'une vue fine sur le comportement du système : réponses produites, sources récupérées, contextes transmis, outils utilisés.

5.3 Notebook d'analyse

Le notebook d'analyse a joué un rôle central. Il a permis de transformer des résultats bruts en diagnostic exploitable : distribution globale des scores, analyse par catégorie de question, repérage des cas faibles, exploration des liens entre qualité de retrieval et qualité de réponse. Cette étape a permis de passer d'une logique "on a des scores" à une logique "on sait où agir".

6. Structuration avec Pydantic et PydanticAI

6.1 Motivation

Pour faire évoluer correctement le projet, il fallait mieux contrôler le format des données manipulées. Dans un système fondé sur des appels LLM, les sorties sont par nature souples — ce qui est utile en prototype, mais problématique quand il s'agit de tracer les exécutions, d'exploiter les réponses en aval ou de les évaluer automatiquement.

6.2 Apport de Pydantic

Pydantic a permis de formaliser les objets principaux du système : documents parsés, chunks, questions utilisateur, contextes récupérés et réponse finale. Ces modèles apportent une validation des données, une standardisation des échanges entre composants, une meilleure lisibilité du code et une exploitation facilitée lors de l'évaluation.

6.3 Apport de PydanticAI

L'introduction de PydanticAI constitue un tournant architectural. Le système n'est plus un pipeline fixe « retrieval → génération », mais un agent outillé capable de choisir dynamiquement le bon outil selon la nature de la question. Cette évolution a permis d'encoder des instructions plus explicites, de centraliser la logique de routage, de conserver une sortie structurée et de tracer l'exécution. PydanticAI a donc servi de support à l'évolution vers une architecture hybride.

7. Introduction du SQL : traitement des données structurées

7.1 Pourquoi le RAG seul ne suffisait plus

Le système initial traitait les tables statistiques comme de simples documents textuels. Cette approche peut fonctionner sur des cas simples, mais elle devient fragile dès qu'une question nécessite un tri, une comparaison, un classement, un filtrage ou une agrégation. Répondre à « Qui a le meilleur pourcentage à 3 points ? » ne relève pas d'une similarité textuelle : c'est une question de structure de données.

7.2 Ingestion de l'Excel en base SQLite

Un pipeline d'ingestion a été mis en place pour transformer le fichier Excel NBA en base SQLite, avec trois tables principales :

- players_stats
- teams
- metric_dictionary

Les statistiques ne sont plus traitées comme du texte approximatif, mais comme une base de faits interrogeable de manière fiable.

7.3 Conception du SQL tool

L'outil SQL construit n'est pas un simple text-to-SQL génératif sans contrainte. Il intègre un schéma explicite, des garde-fous de sécurité, des refus hors périmètre, des règles métier et des validations de requêtes. Ce choix répond à un principe important : dans un système LLM appliqué à des données structurées, la logique métier ne doit pas être entièrement déléguée au modèle. Certains aspects doivent rester sous contrôle déterministe.

7.4 Gestion du hors périmètre

Le SQL tool a été conçu pour savoir refuser certaines questions. Les données relatives au dernier match, à un résultat live, à un transfert récent ou à un événement temps réel ne sont pas couvertes par la base locale. Plutôt que de produire une réponse arbitraire ou hallucinée, le système signale explicitement ces cas et les refuse.

8. Amélioration du retrieval documentaire

8.1 Contexte

L'ajout du SQL ne résout pas tout : certaines questions restent profondément documentaires (débat autour d'un joueur, synthèse d'un post Reddit, définition contextuelle). Améliorer la branche RAG demeurerait donc indispensable.

8.2 Réécriture de requête

Une première amélioration a consisté à réécrire certaines requêtes utilisateur avant de lancer la recherche vectorielle. L'objectif : réduire le bruit conversationnel, conserver les entités importantes et orienter la recherche vers les vrais signaux sémantiques. Une bonne question utilisateur n'est pas toujours la meilleure requête pour un moteur de retrieval dense.

8.3 Reranking heuristique

Un reranking a été ajouté après récupération d'un ensemble de candidats plus large, pour mieux tenir compte du ciblage documentaire explicite, de recouvrements lexicaux, du type de document et de la nature de la question. Cette étape améliore le classement des bons résultats lorsque plusieurs chunks sont sémantiquement proches.

8.4 Chunking différencié

La taille des chunks a été adaptée selon le type de document : un post Reddit, une définition de métrique et un texte long n'ont pas la même granularité utile. Des unités indexées plus cohérentes avec le besoin de la question améliorent mécaniquement la qualité du retrieval.

9. Réévaluation du système amélioré

9.1 Nécessité de la réévaluation

Une amélioration n'a de valeur que si elle peut être observée et mesurée. Après l'introduction de PydanticAI, du SQL tool et des améliorations de retrieval, relancer l'évaluation était indispensable pour répondre à une question concrète : les modifications ont-elles réellement amélioré le système, et dans quelle mesure ?

9.2 Adaptation de l'évaluation au système hybride

Le système final n'étant plus un simple pipeline RAG, les contextes d'évaluation ont été enrichis pour prendre en compte non seulement les chunks RAG, mais aussi une preuve issue du SQL : requête exécutée, aperçu des lignes, nombre de résultats, notes associées. Cette adaptation est nécessaire pour que la métrique faithfulness évalue la réponse à partir de la preuve réellement utilisée par le système, et non d'un contexte partiel.

9.3 Comparaison des runs

Un script de comparaison a été mis en place pour confronter plusieurs runs d'évaluation sur les métriques globales, par question, en repérant améliorations, régressions et cas suspects. Cette

étape transforme l'amélioration du système en objet mesurable plutôt qu'en simple impression qualitative.

10. Industrialisation minimale

10.1 Dockerisation

L'application a été containerisée pour rendre son exécution reproductible. Le conteneur installe l'environnement, copie le projet, expose l'application Streamlit et lance le système dans un environnement isolé. La dockerisation a été validée sur plusieurs cas : démarrage de Streamlit, chargement du vector store, exécution d'un cas RAG, d'un cas SQL et gestion d'un cas hors périmètre.

10.2 Observabilité avec Logfire

Le projet a été instrumenté avec Logfire pour suivre les runs de l'agent, les appels d'outils et les requêtes HTTP associées au modèle. Cette observabilité facilite le débogage, l'analyse des tool calls et la compréhension de la chaîne effective d'exécution.

10.3 Intégration continue minimale

Une première CI GitHub Actions a été mise en place, centrée sur la qualité de code via Ruff. Les tests dépendant d'un appel réel à l'API Mistral n'ont pas été intégrés à la CI principale, afin de ne pas rendre l'intégration continue dépendante d'un service externe et d'un secret API. Il s'agit d'un compromis raisonnable dans notre contexte actuel : la qualité statique du code est vérifiée, les tests d'intégration restent exécutables localement.

11. Résultats quantitatifs de l'évaluation

Cette section présente les résultats chiffrés issus des runs d'évaluation RAGAS conduits sur 50 questions, réparties en six catégories. Trois runs sont comparés : la baseline historique (système V1 initial), la baseline uploadée (système hybride avant optimisations finales) et le dernier run (système final).

11.1 Tableau synthétique des métriques globales

Métrique	Baseline hist.	Baseline upload.	Dernier run	Delta vs hist.
Faithfulness	0.530	0.931	0.954	+80 %
Answer relevancy	0.519	0.825	0.817	+57 %
Hit@k	0.444	0.400	0.422	-5 %
MRR	0.444	0.389	0.422	-5 %
Recall@k	0.344	0.322	0.344	0 %
Precision@k	0.089	0.080	0.084	-5 %
Tool routing OK	—	0.956	0.956	—
Tool call F1	—	0.985	0.985	—

11.2 Analyse globale

Les gains par rapport à la baseline historique sont très significatifs sur les métriques de génération. La faithfulness passe de 0,53 à 0,95, soit une progression de près de +80 % — ce qui traduit une nette amélioration de la fidélité des réponses aux preuves disponibles (documents RAG ou résultats SQL). L'answer relevancy progresse de 0,52 à 0,82 (+57 %), confirmant que les réponses sont beaucoup plus pertinentes vis-à-vis des questions posées.

Ces deux gains sont principalement attribuables à l'introduction du SQL tool et à la structuration via PydanticAI : les réponses aux questions statistiques, qui hallucinaient fréquemment dans la V1, sont maintenant ancrées dans des données vérifiables.

Les métriques de retrieval (hit@k, MRR, precision@k) affichent en revanche de légères régressions par rapport à la baseline historique (de l'ordre de -5 %). Ce résultat s'explique par un changement de périmètre : le système hybride traite désormais les questions SQL sans passer par le retrieval documentaire, ce qui fait mécaniquement baisser le hit@k sur ces questions — sans que cela traduise une dégradation réelle.

Par rapport à la baseline uploadée (système hybride sans optimisations finales), le dernier run consolide les acquis sans régression notable. La faithfulness progresse encore légèrement (+2,5 %), le retrieval s'améliore sur hit@k (+5,6 %) et MRR (+8,6 %). Le routage et le tool call F1 restent stables à des niveaux élevés (0,956 et 0,985 respectivement).

11.3 Résultats par type de question

Le tableau suivant présente les scores détaillés par catégorie de question sur le dernier run (50 questions au total).

Catégorie	N	Faithfulness	Relevancy	Hit@k	Routing OK	F1
numeric_single	10	1.000	0.895	0.000	1.000	1.000
numeric_compare	10	1.000	0.960	0.000	1.000	1.000
hybrid_txt+num	8	0.958	0.880	0.875	0.875	0.958
text_lookup	12	0.920	0.907	1.000	1.000	1.000
out_of_scope	5	0.933	0.000*	—	—	—
noisy_or_ambiguous	5	0.867	0.875	0.000	0.800	0.933

11.4 Lecture par catégorie

Les questions numériques (numeric_single et numeric_compare, 20 questions) obtiennent les meilleurs résultats : faithfulness et tool call F1 à 1,00, answer relevancy supérieure à 0,89. C'est le gain le plus direct de l'ajout du SQL tool — ces questions, qui hallucinaient dans la V1, sont désormais traitées avec une fiabilité maximale.

Les questions de type text_lookup (12 questions documentaires issues des fichiers Reddit) affichent également d'excellents scores : faithfulness à 0,92, relevancy à 0,91, hit@k parfait à 1,00. Le retrieval amélioré et le chunking différencié ont produit leur effet sur cette catégorie.

Les questions hybrides (hybrid_text_plus_number, 8 questions) affichent une faithfulness à 0,96 et une relevancy à 0,88. Le hit@k est bon (0,875), mais révèle que certaines questions complexes

nécessitant à la fois une preuve documentaire et une valeur chiffrée restent un défi de routing — c'est la principale fragilité résiduelle du système.

Les questions hors périmètre (`out_of_scope`, 5 questions) obtiennent une `faithfulness` à 0,93 : le système formule correctement ses refus, ancrés dans une logique explicite. L'`answer relevancy` à 0,00 reflète l'interprétation RAGAS d'un refus comme une non-réponse — ce qui est attendu et non problématique métier.

Les questions bruitées ou ambiguës (`noisy_or_ambiguous`, 5 questions) constituent la catégorie la plus fragile : `faithfulness` à 0,87 et `hit@k` à 0,00, ce qui signifie que le retrieval ne parvient pas à identifier les bonnes sources lorsque la question est formulée de façon imprécise ou combinée. C'est un axe d'amélioration identifié.

12. Résultats et apports du projet

Le projet a permis de construire un système significativement plus riche et plus robuste qu'un assistant RAG simple. Ses apports s'articulent sur plusieurs dimensions :

Dimension	Apport principal
Fonctionnel	Traitement de deux familles de questions : documentaires et statistiques
Méthodologique	Démarche complète : baseline → analyse → amélioration → réévaluation
Architectural	Agent PydanticAI structuré, sortie typée, routage centralisé
Technique	SQL tool : <code>faithfulness</code> de 0,53 → 0,95, questions numériques avec une fiabilité proche des 100 %
Ingénierie	Dockerisation, CI, README détaillé, observabilité Logfire

13. Limites actuelles

Malgré les améliorations apportées, plusieurs limites demeurent et méritent d'être signalées honnêtement.

- Certains tests d'intégration reposent encore sur des appels réels à l'API Mistral, ce qui complique leur intégration directe dans une CI publique.
- Certaines questions mixtes — mobilisant à la fois une dimension statistique et une dimension argumentative — peuvent encore poser des difficultés de routage ou d'interprétation.
- La qualité du système dépend fortement du dataset d'évaluation, de la qualité des données disponibles et du périmètre réel de la base locale.
- Les questions temporelles récentes ou live ne sont pas couvertes par la base SQLite locale : le système doit refuser ou signaler ces cas, mais ne peut pas y répondre.
- Le script de comparaison des runs utilise encore des chemins codés en dur, ce qui limite sa réutilisabilité.

14. Pistes d'amélioration

Plusieurs prolongements naturels peuvent être envisagés pour faire évoluer le projet.

- Séparer explicitement les tests unitaires, les tests d'intégration et les tests dépendant d'un service externe, avec une CI conditionnelle pour les tests Mistral.
- Rendre le script de comparaison des runs entièrement paramétrable en ligne de commande (argparse).
- Enrichir le routage hybride : certaines questions complexes pourraient bénéficier d'une stratégie combinant retrieval documentaire et SQL de manière plus fine.
- Diversifier les catégories de questions dans le dataset d'évaluation pour améliorer la représentativité des métriques.
- Compléter l'instrumentation Logfire sur les validations de modèles et certaines étapes intermédiaires du pipeline.
- Permettre une évaluation des questions par l'utilisateur enregistrable en base de données.
- Enregistrer un historique ré-injectable dans le prompt.

15. Conclusion

Ce projet a permis de faire évoluer un prototype RAG initial vers un système hybride plus structuré, mieux évalué et plus robuste. La démarche adoptée a été délibérément progressive : partir d'un système fonctionnel, mettre en place une évaluation formelle, identifier les limites, introduire une meilleure structuration, ajouter un outil SQL, améliorer le retrieval, réévaluer l'ensemble, puis renforcer la reproductibilité et l'observabilité.

L'intérêt principal du projet réside dans cette articulation entre mise en place technique et évaluation raisonnée. L'objectif n'était pas seulement de faire fonctionner un LLM, mais de construire un système capable de justifier ses réponses, d'utiliser le bon outil selon le contexte et de s'améliorer à partir d'une analyse méthodique de ses limites.

En ce sens, le projet répond bien à l'attendu : évaluer les performances d'un système LLM et l'améliorer à partir d'une démarche structurée, mesurable et documentée.

Annexe — Stack technique

Composant	Technologie
Interface utilisateur	Streamlit
Agent et orchestration	PydanticAI + Pydantic
LLM et embeddings	Mistral AI
Vector store	FAISS
Base de données	SQLite
Évaluation	RAGAS
Observabilité	Logfire

Gestion des dépendances	uv
Qualité de code	Ruff
Déploiement	Docker / Docker Compose
Intégration continue	GitHub Actions
Langage	Python 3.13